Uniwersytet Warszawski
Wydział Matematyki, Informatyki i Mechaniki

Jakub Pieszczek
Nr albumu: 371330

Generatywne modelowanie krótkich muzycznych sampli z użyciem deep learning'u

Praca magisterska
na kierunku INFORMATYKA

Praca wykonana pod kierunkiem
profesor Dominik Ślęzak
Instytut Informatyki

Warszawa, Grudzień 2026

Streszczenie

Synteza dźwięku jest kluczowym elementem produkcji muzyki, tradycjnie wykonywanym poprzez analogowe lub cyfrowe narzędzia implementujące sposoby syntezy takie jak addytywna, subtraktywna czy granularna. Brzmienie pojedyńczego dźwięku jest kształtowane w pożądany sposób za pomocą różnych parametrów, zależnych od stosowanego rodzaju syntezy. Deep Learning pozwala na alternatywny sposób syntezy dźwięku poprzez wyuczone do tego sieci neuronowe. Korzystając z odpowiedniego rodzaju generatywnych modeli, proces ten da się kontrolować za pomocą zestawu parametrów, podobnie do tradycyjnej syntezy. Jednak w przeciwieństwie do niej parametry te są abstrakcyjnymi, wysoko-poziomowymi cechami dźwięku aniżeli ciężkimi do zinterpretowania, nisko-poziomowymi własnościami algorytmów syntezy. Parametry te mogą zostać manualnie wyspecyfikowane poprzez warunkowe uczenie, co wymaga dużego nakładu pracy w oznaczaniu danych, albo wyuczone w nienadzorowany sposób z nieoznaczonych danych. W tej pracy próbuję zastosować do tego zadania Variational Auto-Encodery na trzech różnych, własnoręcznie zebranych zestawach danych - angielskich słowach, brzmieniach perkusyjnych oraz efektów specjalnych - wyciągając z nich parametry w nienadzorowany sposób. Pokazuję, że modele te są w stanie generować dźwięk w 48kHz o wysokiej jakości, oraz że modele te uczą się skompresowanej reprezentacji brzmień w której zakodowane są ich wysoko-poziomowe, intuicyjne własności. Modyfikowanie tej reprezentacji w odpowiedni sposób pozwala na kontrolowanie tych parametrów w syntezowanych dźwiękach. Dodatkowo opisuję sposób w jaki wytrenowany model mógłby zostać opakowany we wtyczkę gotową do użycia w dowolnym cyfrowym środowisku do produkcji muzycznej (DAW).

Słowa kluczowe

VAE, variational auto-encoders, audio modeling, waveform modeling, neural synthesizer

Dziedzina pracy (kody wg programu Socrates-Erasmus)

11.4 Sztuczna inteligencja

Klasyfikacja tematyczna

Computing methodologies Maximum likelihood modeling
Computing methodologies Latent variable models
Computing methodologies Neural networks
Computing methodologies Machine learning

Tytuł pracy w języku angielskim

Generative modeling of short musical samples using deep learning

Spis treści

0.0.1. algorytmy syntezy i kontrola 5

1. Podstawy teoretyczne 9
1.1. Reprezentacja dźwięku 9
1.2. Deep learning i modele generatywne 10
1.2.1. Modelowanie generatywne 10
1.2.2. Sieci Bayesowskie 11
1.2.3. Charakterystyka podejść z użyciem deep learningu 12
1.2.4. Sieci Rekurencyjne 12
1.2.5. GANy 14
1.2.6. VAE 15
1.3. Powiązane prace 17
1.3.1. WaveNet 17
1.3.2. SampleRNN 19
1.3.3. 19
1.4. Normalizing Flows 19
1.5. DDSP 19

2. Architektura 21

3. Eksperymenty oraz ewaluacja 23

4. Podsumowanie i wnioski 25

Wprowadzenie

Synteza dźwięku jest kluczowym elementem produkcji muzyki. Poprzez opracowywanie coraz nowszych metod syntezy, rozwój elektronicznych technologii w drugiej połowie XX wieku przyczynił się do fundamentalnych zmian w tym, jak brzmiała ówczesna muzyka. Na swój sposób czerpała z tego muzyka eksperymentalna - aktywnie szukająca nowych form ekspresji i badająca możliwości artystycznego wykorzystania najnowszych technologii - jak i muzyka popularna, opierająca czasem brzmienie całego nowego gatunku na jednym konkretnym nowym instrumencie.

TODO: enaktywizm

Takim przełomowym instrumentem był na przykład syntezator perkusyjny Roland TR-808. Tuż po wydaniu wcale nie odniósł sukcesu - konkurencyjne maszyny wierniej oddawały prawdziwe brzmienia perkusyjne. Przez swoją niską cenę i charakterstyczne, inne brzmienie, szybko jednak zaczął rozprzestrzeniać się wśród niszowych artystów i stał się bazą do powstającego wtedy hip-hopu i tanecznej muzyki elektronicznej. Podobny los spotkał maszyny TR-909 oraz TB-303, której wydanie The Guardian określił jednym z [50 najważniejszych wydarzeń muzyki tanecznej](https://www.theguardian.com/music/2011/jun/15/tadao-kikumoto-roland).

Nowe sposoby syntezy dźwięku mają fundamentalny wpływ na kształt nowej muzyki nie tylko poprzez swoje unikalne brzmienie. Muzykę kształtuje również sposób - interfejs - w jaki dany instrument pozwala ze sobą wchodzić w interakcję. Tworzenie muzyki jest procesem eksploracji w medium - w używanych narzędziach, instrumentach - i sposoby czy możliwości interakcji z tym medium naturalnie kształtują to, co będziemy w stanie podczas takiej eksploracji wytworzyć. (fragment o musique concrete?) Wywodząca się z Musique Concrete kompozytorka Eliane Radigue poświęciła w ten sposób 25 lat swojej kariery na eksplorację syntezatora ARP 2500, pisząc w latach 1974-1999 prawie wyłącznie kompozycje na nim się opierające. (dron bo ARP 2500? citation needed)

W późniejszych latach, równolegle z coraz to nowszym elektronicznym sprzętem muzycznym, rozwijać zaczęły się software'owe narzędzia do syntezy. Interfejs osobistego komputera dostarczył jeszcze to nowszych sposobów na pracę z dźwiękiem. Syntezatory analogowe zaczęto implementować w wersjach cyfrowych, co pozwoliło na jeszcze większe możliwości kontroli parametrów tej syntezy oraz interakcji między różnymi systemami - syntezatorami, kontrolerami.

0.0.1. algorytmy syntezy i kontrola

Pierwsze instrumenty elektronicze opierały się na sposobie syntezy zwaną *syntezą subtraktywną*. W najogólniejszym sensie polega ona na: 1. wytworzeniu dźwięku za pomocą *oscylatorów* - układów wchodzących w oscylacje harmoniczne, tworzące fale o różnych kształtach (np. sinusoidalna, kwadratowa, piłokształtna) i o określonych częstotliwościach, a następnie 2. przetworzeniu tej fali przez *filtry*, *obwiednie* lub inne układy kształtujące dźwięk. Oscy-

latorów i innych modułów jest zwykle po kilka, połączonych w różne sposoby zależnie od brzmień jakie chce się uzyskać. Najczęściej instrumenty takie mają również klawiaturę na kształt fortepianowej, za pomocą której osoba grająca kontroluje wysokość i długość trwania.

Oprócz długości i wysokości, istotna jest również kontrola samego brzmienia. Tradycyjnie syntezatory umożliwiają to za pomocą różnych pokręteł i przycisków, które bezpośrednio kontrolują ustalone parametry układu od syntezy. Biorąc podstawowy układ z wcześniejszego akapitu, składający się z kilku oscylatorów, filtra i obwiedni - jedno pokrętło może odpowiadać za odcięcie filtra, drugie za rezonans, a zestaw guzików w liczbie odpowiadającej oscylatorom może zmieniać kształt ich fali między trójkątną a piłokształtną. Bardzo często na takich syntezatorach występuje też zestaw pokręteł kontrolujących ostatnią obwiednię kształtującą głośność dźwięku w czasie, z klasycznymi oznaczeniami ąttack", śustain", "decay"i żelease"(stąd nazwa óbwiednia ASDR").

Używającj takiego interfejsu, osoba grająca na syntezatorze zmienia parametry syntezy i kształtuje brzmienie według swojej wizji. Pole do manewru przy tworzeniu takiego interfejsu jest ogromne - oprócz pokręteł czy guzików można wykorzystywać na przykład elementy czułe na nacisk, slidery, akcelerometry, czy nawet wejścia zewnętrznych sygnałów kontrolnych. Różnorodność jest ogromna, a ostateczny wygląd takiego interfejsu i to jak się z niego korzysta bezpośrednio wpływa na to jak będą na nim wyglądały performance'y i jakie brzmienia będzie można wydobyć.

Software'owe narzędzia do syntezy również cieszą się ogromną różnorodnością powstających interfejsów. Cyfrowy algorytm syntezy da się kontrolować nie tylko za pomocą cyfrowych pokręteł czy guzików, ale też za pomocą dowolnego innego sygnału który przetłumaczymy na cyfrowy format. Oznacza to, że możliwe staje się na przykład wykorzystanie czujników ruchu, światła, dźwięku, czy algorytmów wyczytujących informacje z kamery. Taki sygnał następnie można w dowolny sposób podpiąć do algorytmu syntezy, tworząc brzmienie które bezpośrednio kształtują warunki w pomieszczeniu lub czyiś ruch, gesty, czy nawet mimika twarzy.

Osiągnięcia z dziedziny uczenia maszynowego ostatniej dekady znalazły szerokie zastosowanie przy wielu problemach związanych z dźwiękiem. Zdyskretyzowane fale dźwiękowe w naturalny sposób dają się połączyć z uczącymi się algorytmami, a rosnąca ilość dostępnych danych z tej dziedziny pozwala wyciągać z nich bardzo skomplikowane wzorce statystyczne. Uczenie maszynowe jest wykorzystywane z sukcesem przy takich zadaniach przetwarzania sygnału jak synteza mowy [Shen et al., 2018] [Donahue et al., 2021], transkrypcja [Baevski et al., 2020] czy separacja źródeł [Défossez, 2021]. W szczególnym przypadku sygnału dźwiękowego jakim jest muzyka, poruszane problemy to na przykład rozpoznawanie gatunku [Castellon et al., 2021], cech utworu [Castellon et al., 2021] czy generowanie muzyki o danej charakterystyce [Dhariwal et al., 2020].

Modelowanie muzyki jest ciężkim problemem, ponieważ jako sygnał posiada wiele skomplikowanych zależności na wielu skalach czasowych. Poczynając od oscylacji pojedyńczej fali o danej częstotliwości lub zgodności faz różnych częstotliwości, czyli zależności na poziomie milisekund, a kończąc na spójnej formie utworu, który może trwać kilka minut, uchwycenie wszystkich tych własności jednocześnie jest nietrywialnym zadaniem. W związku z tym, badania często skupiają się na konkretnym podzbiorze tych aspektów, korzystając z symbolicznych reprezentacji abstrahujących niepotrzebne szczegóły. Prace mające na celu generować dłuższe kompozycje często korzystają na przykład z MIDI [1] [Huang et al., 2018]. Jest to format reprezentujący ciąg nut o określonej wysokości, głośności oraz określonym początku i końcu. Nie zawiera informacji o tym na jakim instrumencie nuty są grane ani żadnych szczegółów ich wykonania (na przykład sposób uderzenia w przypadku instrumentów strunowych), co

[1] https://en.wikipedia.org/wiki/MIDI

pomaga w skupieniu się na bardziej wysokopoziomowych własnościach kompozycji.

Aby modelować niskopoziomowe własności dźwięku, jak na przykład barwę, trzeba operować bezpośrednio na poziomie pojedyńczych próbek. Takie podejście jest o wiele bardziej wymagające obliczeniowo, gdyż sygnały dźwiękowe, w celu utrzymania akceptowalnej jakości, próbkowane są zwykle z częstotliwością co najmniej 16 kHz. Oznacza to, że kilkusekundowe nagranie może mieć już kilkadziesiąt tysięcy próbek, co znacznie utrudnia zadanie uchwycenie zależności na całej długości. Niemniej jednak w ostatnich lat, w związku z postępującymi zasobami obliczeniowymi i coraz pojemniejszymi modelami, sposób ten jest coraz częściej eksplorowany. Jednym z zadań w którym takie podejście odnosi sukcesy jest synteza mowy, gdzie modele takie jak na przykład WaveNet [van den Oord et al., 2016a] generują realistycznie brzmiące nagrania autoregresywnie modelując sygnał próbka po próbce.

Cel pracy

Celem tej pracy jest modelowanie krótkich, pojedyńczych dźwięków (tzw. *sampli*, jak na przykład pojedyńcze uderzenie klawisza na fortepianie) w ten sposób, żeby móc generować nowe i wiarygodne sample przy jednoczesnym utrzymaniu wysokiej jakości dźwieku, porównywalnej z najnowszymi pracami w tym zakresie. Praca nie obejmuje takich aspektów muzycznych jak harmonia czy aranżacja, skupiając się głównie na barwie. Wykorzystane zostały trzy zestawy danych:

- VCTK [Veaux et al., 2017], przeprocesowany tak
- Percussive - manualnie zebrane perkusyjne brzmienia z ogólnodostępnych paczek
- SFX - manualnie zebrane dźwięki z biblioteki Freesound oznaczone jako efekty specjalne

Generatywne modele mają szeroki potencjał jako narzędzia kreatywne w procesie tworzenia muzyki. Przykładem wdrożonego już rozwiązania tego typu jest zestaw wtyczek od Google'owego projektu Magenta, Magenta Studio [Google, 2021]. Jest to pięc wtyczek do Ableton'a [2] (który jest tzw. *Digital Audio Workstation*, czyli oprogramowaniem do tworzenia muzyki), które pozwalają użytkownikowi korzystać z modeli opracowanych w Magenta i aplikować je do własnych sekwencji MIDI (reprezentujących jakąś melodię lub motyw perkusyjny). Wtyczka „Continue", na przykład, bierze sekwencję i kontynuuje ją w realistyczny i zróżnicowany sposób, tworząc sekwencję nawet kilkukrotnie dłuższą niż oryginalna. Innym takim narzędziem od Magenty jest NSynth Super [Google, 2018]. Jest to tak właściwie elektroniczny instrument, który pozwala na żywo interpolować barwę dźwięku między czterema wybranymi instrumentami wykorzystując do tego *embeddingi* (rzutowania na przestrzeń o mniejszej wymiarowości) z modelu NSynth [Engel et al., 2017].

Sample, obok syntezatorów czy efektów, są jednym z podstawowych budulców przy produkcji muzyki elektronicznej. Po uprzednim dostosowaniu ich do swoich potrzeb za pomocą odpowiednich narzędzi, układane są z nich bardziej skomplikowane kompozycje. Selekcja tych sampli ma znaczący wpływ na późniejsze brzmienie i charakter utworu. Opracowanie modeli generujących nowatorskie i dobrze brzmiące sample może również mieć potencjalne zastosowania w tym procesie, dając producentom na przykład coś w rodzaju syntezatora do generowania sampli odpowiadających ich wymaganiom. Z myślą o takim konkretnym zastosowaniu stworzony został model nauczony na zestawie werbli.

[2] `https://www.ableton.com/`

Struktura pracy

Praca jest podzielona na cztery części. W pierwszym rozdziale omówione są podstawy teoretyczne i poprzednie prace z dziedziny na których ta się opiera, czyli:

- deep learning i modelowanie generatywne,
- najważniejsze poprzednie prace w dziedzinie,
- estymowanie skomplikowanych rozkładów prawdopodobieństwa za pomocą *Normalizing Flows*, oraz
- DDSP.

W drugim rozdziale opisana jest zastosowana architektura modelu oraz zestawy danych. W trzecim rozdziale przedstawione są wyniki eksperymentów, a w czwartym rozdziale są one podsumowane oraz przedyskutowane są potencjalne zastosowania i dalszy możliwy kierunek eksploracji w tym kierunku.

Rozdział 1

Podstawy teoretyczne

1.1. Reprezentacja dźwięku

Dźwięk może być reprezentowany cyfrowo na wiele sposobów. Pierwszym z nich, najbardziej podstawowym, jest ciąg próbek przedstawiający zdyskretyzowaną falą dźwiękową. Ludzki słuch nie wyłapuje częstotliwości powyżej około 20 kHz, więc z twierdzenia Nyquista-Shannona [Shannon, 1949] wiemy, że jesteśmy w stanie zdyskretyzować falę bez straty informacji o ile zrobimy to z wystarczająco dużą częstotliwością - co najmniej dwa razy większą od najwyższej częstotliwości którą chcemy uchwycić. Z tego powodu wszelkie oprogramowanie przetwarzające dźwięk posługuje się najczęściej częstotliwością próbkowania 48 kHz. W szczególnych przypadkach używane są też 96 kHz lub nawet 192 kHz, co redukuje ryzyko aliasingu [1]. W kontekscie modelowania, wysoka częstotliwość próbkowania oznacza jednak dłuższy ciąg danych wejściowych do modelu. Mniejsza częstotliwość wiąże się z degradacją jakości, więc w ramach kompromisu najczęściej w dziedzinie używane są częstotliwości od 16 kHz do 24 kHz. Pomijany jest w ten sposób duży, słyszalny zakres wysokich częstotliwości, jednak najważniejsze dla ucha są częstotliwości średnie (do 8 kHz), więc jest to akceptowalna strata w przypadku więszkości problemów które są poruszane.

Inną często używaną reprezentacją jest spektrogram, czyli wykres zmian częstotliwości składowych sygnału w czasie. Spektrogram otrzymuje się z sygnału dźwiękowego za pomocą krótkookresowej transformaty Fourier'a (STFT). Przedstawia on dla każdego kroku (ang. *frame*) w czasie rozkład natężenia częstotliwości składowych, gdzie długość jednego kroku jest zależna od parametrów transformaty. Technicznie rzecz biorąc, wartości w spektrogramie są liczbami zespolonymi zawierającymi również informację o fazie każdej składowej. Ponieważ nie przekłada się ona bezpośrednio na ludzką percepcję, najczęściej pomija się ją biorąc normę tych wartości. Słuch ludzki postrzega zmiany zarówno w częstotliwości jak i w natężeniu logarytmicznie, więc oś częstotliwości spektrogramu jest często przeskalowywana używając skali Mel [Stevens et al., 1937]. Skala ta została określona na podstawie pomiarów empirycznych i odpowiada temu jak ludzie postrzegają zmiany w częstotliwości. Wartości spektrogramu, czyli natężenie, również są często skalowane logarytmicznie, jednak nie ma tutaj powszechnej formuły.

Oprócz reprezentacji kodujących bezpośrednio to, jak wygląda sygnał dźwiękowy, istnieje też wiele reprezentacji symbolicznych. Konkretna forma takiej reprezentacji jest już zależna od funkcji jaką ma pełnić i dziedziny jakiej dotyczy. Żeby zapisać na przykład w jaki sposób wymawiane są pewne wyrazy używa się międzynarodowego alfabetu fonetycznego [2]. Żeby spi-

[1] https://en.wikipedia.org/wiki/Aliasing

[2] `https://en.wikipedia.org/wiki/International_Phonetic_Alphabet`

sać utwór muzyczny w celu późniejszego wykonania przez solistę bądź zespół, używa się zapisu nutowego. Blisko powiązany z zapisem nutowym jest wcześniej już przytoczony format MIDI, będący lepiej przystosowany do odczytu komputerowego. Zapisy symboliczne abstrahują wiele aspektów konkretnego dźwięku który mogą reprezentować. W przypadku wykonania utworu z zapisu nutowego, pominięte aspekty mające wpływ na końcowe brzmienie to między innymi:

- interpretacja wykonawcy (szczegóły takie jak akcenty, frazowanie, tempo/zmiany w tempie, sposób wydobycia dźwięku z instrumentu),
- akustyka pomieszczenia, czy
- instrument na którym jest wykonywany.

Zaletą tego jest jednak o wiele mniejsza zawartość informacji w takim zapisie, co pozwala na jego efektywniejsze modelowanie. Przykładowo, przy generowaniu melodii nie potrzebujemy uwzględniać żadnego z powyższych, a jedynie wysokość i długość każdej nuty. Użycie w takim przypadku MIDI jest najlepszym rozwiązaniem.

Często do rozwiązania jakiegoś problemu używane jest kilka reprezentacji naraz. System syntezy mowy Google'a Tacotron 2 [Shen et al., 2018] używa pośrednich reprezentacji - najpierw przekształca wejściowy tekst w spektrogram, aby potem ze spektrogramu generować dźwięk. WaveNet [van den Oord et al., 2016a] używa wysokopoziomowych, wcześniej wyekstrahowanych reprezentacji - ciągu fonemów i częstotliwości bazowej - jako sygnału warunkującego do generowania mowy. Wave2Midi2Wave od Magenty [Hawthorne et al., 2019] tworzy hierarchiczną strukturę generującą najpierw sekwencje MIDI, która jest następnie sygnałem warunkującym dla modelu opartego na WaveNet generującego wykonanie tej sekwencji na fortepianie.

1.2. Deep learning i modele generatywne

1.2.1. Modelowanie generatywne

W klasycznym ujęciu, uczenie statystyczne polega na modelowaniu interesujących nas zmiennych przewidywanych Y jako funkcji zmiennych obserwowanych X (zwanych często *cechami*, z angielskiego *features*) - $Y = f(X)$, gdzie parametry funkcji f uczy się na podstawie zestawu danych treningowych. Jeśli dziedziną zmiennych Y są liczby rzeczywiste, problem ten nazywa się regresją, a jeśli jest to jakiś dyskretny zbiór - klasyfikacją. W przypadku klasyfikacji zamiast deterministycznej zależności $Y = f(X)$ częstszym wyborem jest modelowanie rozkładu warunkowego $P(Y|X)$. Takie sformułowanie ma dużo zalet - pozwala nam określić pewność klasyfikatora co do jego predykcji, wstrzymując się na przykład od decyzji jeśli pewność jest za mała lub propagując tę informację dalej w systemie. Metody do estymowania takich rozkładów można podzielić na dwa typy:

- dyskryminatywne - bezpośrednio modelujące $P(Y|X)$, na przykład sieci neuronowe, regresja logistyczna czy CRFy (*Conditional Random Fields* [Lafferty et al., 2001]), oraz
- generatywne - modelujące łączny rozkład $P(Y, X)$, z którego następnie uzyskuje się rozkład warunkowy $P(Y|X)$, jak na przykład Naive Bayes czy GMMy (*Gaussian Mixture Model*).

Interesującą własnością unikalną dla podejścia generatywnego jest fakt, że przy okazji modelowania zmiennych Y modelujemy również sam rozkład zmiennych wejściowych X. Jego

znajomość pozwala na generowanie nowych, syntetycznych przykładów $\langle x, y \rangle$ bądź obliczenia prawopodobieństwa konkretnego przykładu $P(X = x)$ niezależnie od Y. Znajduje to zastosowanie przy takich zadaniach jak na przykład (TODO generative modeling tasks).

Oprócz typowego nadzorowanego pojdeścia do uczenia statystycznego (ang. *supervised learning*), gdzie na wejściu dane są pary $\langle x, y \rangle$ zmiennych obserwowanych X i przewidywanych Y, istnieje szeroka klasa metod uczenia nienadzorowanego (ang. *unsupervised learning*) wymagająca na wejściu przykładów wyłącznie ze zmiennymi obserwowanymi X. Metody generatywne w tym przypadku mają na celu modelować rozkład tych danych - $P(X)$. Przykładowe zadania w których korzysta się z tego typu rozwiązań to (TODO unsupervised genertive modeling tasks). Dalsza część tego podrozdziału zostanie poświęcona bardziej szczegółowemu opisowi tego podejścia, - wybranych metod oraz różnych problematycznych aspektów - gdyż odpowiada ono problemowi który porusza ta praca.

TODO: DENSITY ESTIMATION?

1.2.2. Sieci Bayesowskie

Ogólną klasą modeli probabilistycznych zaliczających się jako generatywne są sieci bayesowskie. Formalnie sieć bayesowska to skierowany graf acykliczny, w którym wierzchołki odpowiadają zmiennym losowym a krawędzie wskazują na zależności między nimi. Taki graf przedstawia faktoryzację łącznego prawdopodobieństwa wszystkich zmiennych

$$P(X_1, \ldots, X_n) = \prod P(X_i | pa(X_i)) \tag{1.1}$$

gdzie $pa(X_i)$ (od *parents*) to zbiór wierzchołków z których wychodzi krawędź do X_i. Rozkłady każdej zmiennej (tzw. rozkłady lokalne) mogą być dowolne, w szczególności nie muszą być takie same. Przykładowym modelem może być sieć przedstawiającą wpływ wybranych czynników zewnętrznych na prawdopodobieństwo choroby oraz symptomów - patrz rysunek TODO. Ponieważ w pełni wyspecyfikowany model pozwala policzyć łączny rozkład prawdopodobieństwa, jest on przykładem modelu generatywnego. Taki model pozwala również liczyć wszelkie inne interesujące nas rozkłady, czyli rozkłady zwykłe lub warunkowe wybranych zmiennych - proces ten określa się inferencją na drzewie.

W rzeczywistych modelach lokalne rozkłady mają najczęściej nieznane parametry, a celem jest dopasowanie tych parametrów do jakichś zaobserwowanych danych. Używa się do tego zwykle metod opartych na maksymalizacji wiarygodności danych (ang. *maximum likelihood estimation*). Zaobserwowane dane często nie zawierają każdej zmiennej - w takim przypadku zmienne dzieli się na *obserwowane* i *ukryte*. Zwykłe metody maksymalizacji wiarygodności nie dadzą się w takim przypadku zastosować, ponieważ nie da się bez wszystkich zmiennych policzyć funkcji wiarygodności. Klasyczną metodą na uczenie parametrów w takim grafie jest algorytm *Expectation-Maximization* [Dempster et al., 1977], który iteracyjnie na zmianę liczy wartość oczekiwaną funkcji wiarygodności przy założeniu poprzedniego zestawu parametrów oraz parametry maksymalizujące poprzednio wyliczoną funkcje wiarygodności. Taki proces potwarza się aż zbiegnie do satysfakcjonującego wyniku - zostało udowodnione, że zbiega do lokalnego minimum.

Mając w pełni wyspecyfikowaną sieć bayesowską, można jej użyć do odpowiedzi na probabilistyczne pytania na temat modelowanych zmiennych. Częstym przypadkiem jest sytuacja, w której zaobserwowany został jakiś podzbiór zmiennych. Pytanie może wtedy być, pod warunkiem zaobserwowanych wartości, o rozkład wybranego pozdbioru zmiennych. Proces liczenia takiego rozkładu przy zaobserwowanych wartościach nazywa się inferencją w danym grafie. Jest to przykład inferencji bayesowskiej - dany jest rozkład początkowy (ang. *prior di-*

stribution), który następnie aktualizuje się po obserwacji danych (ang. *evidence*) żeby uzyskać nowy rozkład (ang. *posterior distribution*).

1.2.3. Charakterystyka podejść z użyciem deep learningu

Istnieją trzy główne rodzaje deep learningowych modeli generatywnych:

- modele autoregresywne,
- GANy, oraz
- VAE.

Rodzaje te, wraz z konkretnymi przykładami, zostaną omówione w następnych sekcjach.

Jedną z możliwych charakteryzacji modeli generatywnych jest sposób w jaki pokrywają docelowy rozkład. W sytuacji, gdzie nie są w stanie uchwycić jego pełnej zmienności, mogą zachować się w różny sposób. Wyróżnia się tutaj dwa typy modeli - *mode-seeking* oraz *mode-covering*. W pierwszym przypadku model próbuje dokładnie pokryć jakiś mniejszy podzbiór modelowanej przestrzeni („wyszukuje" go), ignorując inne potencjalnie liczne obszary. Przykładem tego są GANy, podatne na tzw. *mode collapse*, czyli ekstremalne sytuacje tego zachowania. W drugim przypadku model próbuje pokryć cały rozkład, przez co poszczególne obszary mogą być odwzorowane z małą dokładnością, jednak lepiej uchwycona jest ogólna charakterystyka wejściowego zbioru danych. Modele oparte na maksymalizowaniu funkcji wiarygodności zwykle zachowują się w ten sposób, czyli na przykład VAE lub modele autoregresywne.

Inną ważną własnością modeli generatywnych jest to, czy jest się w stanie odzyskać dla danego przykładu x jego dokładne prawdopodobieństwo $p(x)$. Modele *implicite* - na przykład GANy czy VAE- na to nie pozwalają, dając wyłącznie możliwość ciągnięcia nowych próbek z wyestymowanego rozkładu (stąd nazwa - rozkład jest zawarty w modelu implicite). W niektórych przypadkach - jak na przykład VAE - $p(x)$ da się estymować za pomocą technik samplowania z nieznanych rozkładów. Modele które bezpośrednio pozwalają dla danego x policzyć $p(x)$ nazywa się modelami *explicite* - są to na przykład modele autoregresywne, lub bardziej klasyczne techniki estymowania rozkładów.

Przy niektórych zastosowaniach, jak na przykład synteza mowy, ważna jest również prędkość generowania przykładów. Modele autoregresywne są ze swojej natury bardzo wolne - w przypadku dźwięku, gdzie może być kilkanście próbek na sekundę, każda z nich musi być generowana po kolei. GANy są za to przykładem architektury, w której generowanie każdej z próbek da się zrównoleglić, przez co generowanie kolejnych przykładów jest odpowiednio szybsze.

1.2.4. Sieci Rekurencyjne

Podstawowym narzędziem deep learningu do modelowania sekwencji są sieci rekurencyjne (ang. *Recurrent Neural Network*). Jest to szeroka klasa architektur charakteryzująca się połączeniami rekurencyjnymi - idącymi z późniejszych warstw do wcześniejszych. W praktyce oznacza to, że taka sieć zachowuje pewien stan wewnętrzny (ang. *hidden state*) przekazywany między obliczeniami na każdym kolejnym elemencie sekwencji, pozwalając jej na uchwycenie złożonych zależności na przestrzeni wielu kroków. Wyjściem jest przetworzona sekwencja tej samej długości oraz końcowy stan wewnętrzny - w zależności od zadania wykorzystuję się całość wyjścia lub jego część. Na ilustracji przedstawiony jest schemat najbardziej podstawowej, jednowarstwowej sieci rekurencyjnej. Architektury tego typu osiągają sukcesy przy wielu zadaniach, jak na przykład generowanie tekstu [Sutskever et al., 2011] [Graves, 2014],

ręcznego pisma [Graves, 2014], tłumaczenie maszynowe [Kalchbrenner and Blunsom, 2013], czy modelowanie danych z czujników ruchu [Sutskever et al., 2008].

Sieci rekurencyjne naturalnie wpasowywują w schemat autoregresywnych modeli generatywnych - na taką sieć można spojrzeć jako na rozkład warunkowy kolejnego elementu sekwencji pod warunkiem wszystkich poprzednich. Przykładem autoregresywnego modelu dźwięku wykorzystującego sieci rekurencyjne jest SampleRNN [Mehri et al., 2017], który zostanie omówiony w sekcji o powiązanych pracach.

Sieci rekurencyjne w podstawowej wersji, choć teoretycznie są w stanie uchwycić dowolne zależności między elementami sekwencji, w praktyce są podatne na problemy z uczeniem się zależności w długiej skali czasowej z powodu zaniku gradientów. Jedną z popularnych architektur mającą poradzić sobie z tym problemem i osiagającą imponujące wyniki są tzw. sieci *LSTM* (*Long short-term memory*). na obrazku TODO przedstawiony jest schemat komórki rekurencyjnej LSTM. Pomysłem jest zaprojektowanie sieci w ten sposób, żeby mogła dokładniej regulować przepływ informacji zawartej w stanie wewnętrznym. Dokładniej rzecz biorąc, stan ukryty jest złożony z dwóch wektórow - h_t (*hidden state*) oraz c_t (*cell state*) - i trzy „bramki" (warstwy) określają jak ten stan zostanie użyty: *input gate* (i_t), *forget gate* (f_t) oraz *output gate* (o_t). Następujące równiania określają wyjście komórki LSTM dla kroku t przy danym h_{t-1} i c_{t-1}:

$$
\begin{aligned}
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
c_t &= f_t \circ c_{t-1} + i_t \circ \tanh(W_c x_t + U_c h_{t-1} + b_c) \\
h_t &= o_t \circ \tanh(c_t)
\end{aligned}
$$

gdzie σ to funkcja sigmoidalna, a $\circ$ to mnożenie po współrzędnych. Intuicyjnie rzecz biorąc, „bramki" mają następujące funkcje:

- *input* określa ile informacji z wejścia dodać do c_t,
- *forget* określa ile informacji zachować z poprzedniego c_{t-1}, a
- *output* określa co z nowopowstałego c_t przekazać dalej jako h_t.

Taka struktura powoduje, że sieć jest w stanie, jeśli chce, długoterminowo przechowywać w c_t potrzebne informacje. Na przestrzeni lat powstało wiele wariantów architektury LSTM, wykorzystywanych z sukcesami przy wielu problemach. Wykorzystują one na przykład dodatkowe połączenia, wykorzystując c_t do obliczania f_t czy i_t, lub zmieniają funkcje aktywacji (sigmoid/tanh) na inne. Przykładem niedawno opublikowanej, imponującej pracy wykorzystującej jako jeden z głównych komponentów LSTMy to system *OpenAI Five* grający w *Dota 2* [3] - grę typu MOBA [4].

Inną wariacją na temat sieci rekurencyjnych, mającą na celu usprawnienie uczenia się długoterminowych zależności, jest tzw. *GRU* (*Gated Recurrent Unit*). Jest to architektura prostsza niz LSTMy, mająca podobny zamysł i osiągajaca często równie dobre wyniki. w W przeciwieństwie do LSTM, ma tylko jeden wektor stanu ukrytego, h_t, i dwie „bramki" - *update gate* (z_t) i *reset gate* (r_t). Na obrazku TODO przedstawiony jest schemat pojdeyńczej

[3] `https://www.dota2.com/home`

[4] `https://en.wikipedia.org/wiki/Multiplayer_online_battle_arena`

komórki. Następujące równiania określają jej działanie:

$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \\
h_t &= (1 - z_t) \circ h_{t-1} + z_t \circ \tanh(W_h x_t + U_h(r_t \circ h_{t-1}) + b_h)
\end{aligned}
$$

W tym przypadku *update gate* określa (podobnie jak *forget* i *input* przy LSTMach) ile stanu wewnętrznego zachować a ile wziąć z nowo policzonego, a *reset gate* decyduje ile „zapomnieć" z poprzedniego stanu. Podobnie jak przy LSTMach, istnieje wiele wariacji na temat GRU, modyfikujących ilość bramek, połączenia i funkcje aktywacji. Przykładem problemu, w którym GRU osiąga dobre wyniki to transkrypcja tekstu [Ravanelli et al., 2018].

1.2.5. GANy

Bardzo popularnym i osiągającym znakomite wyniki na przestrzeni ostatnich lat rodzajem modeli generatywnych są GANy (*Generative Adversarial Networks*). Wprowadzone w 2017 roku przez Ian'a Goodfellow'a, polegają na przeciwstawieniu sobie dwóch sieci w grze o sumie stałej [5] i uczeniu ich jednocześnie [Goodfellow et al., 2014]. W oryginalnej wersji przeznaczone są do uczenia nienadzorowanego, czyli modelowania samego rozkładu danych $p(x)$ bez zmiennych docelowych Y.

Niech p_x oznacza prawdziwy rozkład z którego pobrane są dane treningowe. Pierwsza z sieci - G, zwana generatywną (ang. *generative network*) - ma za zadanie syntezować jak najbardziej wiarygodne przykłady przekształcając losowe, niskowymiarowe wektory z z wybranego rozkładu p_z (na przykład k-wymiarowego normalnego) na przykłady podobne tym z p_x. W przypadku obrazów, często używa się do tego transponowanych konwolucji. Druga sieć - D, zwana dyskryminującą (ang. *discriminative network*) - ma za zadanie rozróżniać czy przykłady na wejściu zostały stworzone przez sieć generatywną czy są rzeczywistym przykładem z p_x. Sieć G uczy się minimalizować prawdopodobieństwo wykrycia przez D minimalizując $log(1 - D(G(z)))$, zaś D uczy się poprawnie odróżniać przykłady maksymalizując poprzednie wyrażenie oraz $logD(x)$ (wartość $D(x)$ oznacza prawdopodobieństwo, że x pochodzi z p_x). Razem daje to następującą funkcję straty (G) / zysku (D):

$$
V(D, G) = \mathbb{E}_{x \sim p_x}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] \tag{1.2}
$$

którą optymalizuje się (minimalizuje / maksymalizuje) na zmianę względem D oraz G, używając standardowo *stochastic gradient descent*.

W zaledwie kilka lat po oryginalnym sformułowaniu GANy były w stanie uczynić znaczący postęp w dziedzinie generowania syntetycznych obrazów [Karras et al., 2019] [Karras et al., 2020], z modyfikacjami umożliwiającymi rzeczy takie jak transfer stylu [Abdal et al., 2019] czy warunkowe generowanie obrazów [Zhu et al., 2020]. Ich nowatorski paradygmat generatywny wiąże się z między innymi następującymi zaletami:

- GANy są w stanie estymować „ostre" rozkłady prawdopodobieństwa, w przeciwieństwie do metod takich jak VAE czy łańcuchy Markova,
- D i G mogą być zaimplementowane przez dowolną architekturę, oraz
- nie ma ograniczeń nałożonych na rozmiar ukrytej zmiennej z.

[5] `https://en.wikipedia.org/wiki/Zero-sum_game`

Dzięki nim, GANy są w stanie generować bardzo realistyczne obrazy, chociaż ciągle badanym problemem zostaje wcześniej omówiony *mode collapse* powodujący czasem niską różnorodność generowanych przykładów [Wang et al., 2020].

GANy znalazły również zastosowanie w innych dziedzinach niż *Computer Vision*, w szczególności przy generowaniu dźwięku. Jedną z pierwszych prac próbujących zaaplikować GANy do syntezy dźwięku był WaveGAN [Donahue et al., 2019] opierający się na pomyśle, żeby spektrogramy traktować jak obrazy. GANy są sukcesem wykorzystywane są przy syntezie mowy [Kumar et al., 2019] [Donahue et al., 2021] czy śpiewu [Sankaran et al., 2021], gdzie również przejawia się ich zaleta w postaci bardziej realistycznych przykładów, potencjalnie kosztem róznorodności. Najpodobniejszym do tego, co próbuje osiągnąć ta praca (aczkolwiek nie od strony architektury), jest GANSynth [Engel et al., 2019]. Stworzony przez oryginalnych autorów NSynth, osiąga jeszcze lepszą jakość rekonstrukcji instrumentalnych sampli niż pierwotne rozwiązanie oparte na autoenkoderach. Podobnie jak WaveGAN, GANSynth generuje logarytmiczne spektrogramy, estymując przy tym również *instantaneous frequency* [6] w każdym punkcie. Dodatkowo na wejściu do generatora dane są wektory kodujące wysokość oraz natężenie dźwięku, co umożliwia niezależną kontrolę tych parametrów przy generacji.

TODO: anything more?

1.2.6. VAE

VAE (Variational Auto-encoder) to architektura wywodząca się z połączenia auto-enkoderów i nowatorskiej metody inferencji wariacyjnej, zaproponowana w [Kingma and Welling, 2014] i służąca do nienadzorowanego generatywnego modelowania.

Wariacyjne metody inferencji

Niech dana będzie sieć bayesowska. $z \in Z$ to zmienne ukryte (do których standardowo można zaliczyć też parametry rozkładów θ), zaś $x \in X$ to zmienne obserwowane. Celem może być

- optymalizacja parametrów,
- wyestymowanie marginalnego prawdopodobieństwa zaobserwowanych danych, lub
- inferencja na zmiennych ukrytych.

Do każdego z nich wymagana jest znajomość rozkładu $p(z|x)$ - tzw. *posterior distribution*. Często, przy bardziej skomplikowanych zależnościach w modelu, rozkład ten jest niepoliczalny. Wyprowadza się wtedy jego aproksymację - tzw. *approximate posterior* oznaczany zwykle $q(z|x)$ - zakładając najczęściej, że prawidzwy *posterior* można na jakiś sposób sfaktoryzować. Daje to możliwość przybliżonego obliczenia docelowych wielkości, przy czym jakość wyniku zależy od tego jak dobrze *approximate posterior* przybliża rzeczywisty rozkład $p(z|x)$. Metody wykorzystujące takie aproksymacje nazywa się wariacyjnymi metodami inferencji.

Mając dany *approximate posterior* $q(z|x)$, da się wyprowadzić następujący wzór na marginalne prawdopodobieństwo $p(x)$ korzystając z definicji dywergencji Kullbacka-Leiblera [7] $D_{KL}(q||p)$:

$$\log p(x) = D_{KL}(q(z|x)||p(z|x)) + \mathbb{E}_q[-\log q(z|x) + \log p(x,z)] \tag{1.3}$$

[6] https://en.wikipedia.org/wiki/Instantaneous_phase_and_frequency

[7] https://en.wikipedia.org/wiki/Kullback-Leibler_divergence

Ponieważ dywergencja ta jest zawsze większa lub równa zeru, drugi człon sumy jest dolną granicą $\log p(x)$:

$$\log p(x) \geq \mathcal{L}(q, x) = \mathbb{E}_q[-\log q(z|x) + \log p(x, z)] \tag{1.4}$$

i można ją zapisać jako

$$\mathcal{L}(q, x) = -D_{KL}(q(z|x)||p(z)) + \mathbb{E}_q[\log p(x|z)] \tag{1.5}$$

Granicę $\mathcal{L}(q, x)$ nazywa się *Evidence Lower Bound*, lub w skrócie ELBO. Pozwala ona na przybliżenie rzeczywistego marginalnego prawdopodobieństwa zaobserwowanych danych używając *approximate posterior*, przy czym z równania 1.3 widać, że jakość przybliżenia jest tym lepsza, im mniejsze jest $D_{KL}(q||p)$, czyli im podobniejszy jest *approximate posterior* do prawdziwego rozkładu.

Stochastic Gradient Variational Bayes

VAE opierają się na specyficznym, oryginalnym sposobie przeprowadzenia wariacyjnej inferencji. Niech dany będzie model taki jak na obrazku TODO, przedstawiający rozkład $p_\theta(x, z) = p_\theta(x|z)p_\theta(z)$. Załóżmy, że:

- niepoliczalne są $p_\theta(x)$, *posterior* $p(z|x)$ oraz jakiekolwiek jego klasyczne przybliżenia wariacyjne zakładające jakąś faktoryzację - taka sytuacja jest możliwa przy bardziej skomplikowanych rozkładach $p_\theta(x|z)$, modelowanych na przykład przez sieć neuronową, oraz
- zbiór danych jest na tyle duży, że użycie metod opartych na samplowaniu takich jak na przykład *Monte Carlo EM* jest niepraktyczne, a do pojedyńczego kroku optymalizacji najlepiej by móc użyć niewielkiej liczby przykładów, czy nawet jednego.

Autorzy, na wzór metod wariacyjnych, wprowadzają parametryzowany model $q_\phi(z|x)$ mający być przybliżeniem $p(z|x)$. W przeciwieństwie do klasycznych metod, nie zakłada on jednak żadnej faktoryzacji i nie musi mieć żadnej konkretnej formy - może to być również na przykład sieć neuronowa. Parametry ϕ tego modelu mają być docelowo wyuczone wraz z parametrami θ właściwego modelu.

Do uczenia potrzebne jest różniczkowanie granicy $\mathcal{L}(q, x)$ względem ϕ i θ. Typowy estymator takiej pochodnej względem rozkładu $q_\phi(z|x)$ - $\nabla_\phi \mathbb{E}_{q_\phi(z)}[f(z)] = \mathbb{E}_{q_\phi(z)}[f(z)\nabla_{q_\phi(z)} \log q_\phi(z)] \simeq \frac{1}{L}\sum_{l=1}^{L} f(z)\nabla_{q_\phi(z_l)} \log q_\phi(z_l)$, gdzie $z^{(l)} \sim q_\phi(z|x)$ - nie nadaje się do tego zastosownia z powodu zbyt dużej wariancji. W związku z tym autorzy wyprowadzają estymator dolnej granicy nazwany *Stochastic Gradient Variational Bayes* (SGVB). Opiera się on na tym, że zmienną $\tilde{z} \sim q_\phi(z|x)$, przy niezbyt wymagających założenich dla rozkładu q, da się przedstawić jako $\tilde{z} = g_\phi(\epsilon, x)$, gdzie $\epsilon \sim p(\epsilon)$ i g jest różniczkowalne. Estymator wartości oczekiwanej wygląda wtedy następująco:

$$\mathbb{E}_{q_\phi(z|x)}[f(z)] = \mathbb{E}_{p(\epsilon)}[f(g_\phi(\epsilon, x))] \simeq \frac{1}{L}\sum_{l=1}^{L} f(g_\phi(\epsilon_l, x)) \tag{1.6}$$

Aplikując to do definicji z równania 1.4, wychodzi $\tilde{\mathcal{L}}(q, x) \simeq \mathcal{L}(q, x)$:

$$\begin{aligned}
&\widetilde{\mathcal{L}}(q, x_i) = \frac{1}{L}\sum_{l=1}^{L} \log p_\theta(x_i, z_{i,l}) - \log q_\phi(z_{i,l}|x_i) \\
&z_{i,l} = g_\phi(\epsilon_{i,l}, x_i) \\
&\epsilon_{i,l} \sim p(\epsilon)
\end{aligned}$$

Alternatywnie, ponieważ dywergencję $D_{KL}(q(z|x)||p(z))$ da się często obliczyć analitycznie, można wyprowadzić estymator z równania 1.5, dający wersję z jeszcze mniejszą wariancją:

$$\widetilde{\mathcal{L}}(q, x_i) = -D_{KL}(q_\phi(z|x_i)||p_\theta(z)) + \frac{1}{L}\sum_{l=1}^{L} \log p_\theta(x_i|z_{i,l}) \tag{1.7}$$

Mając taki estymator dla pojedyńczego przykładu x_i, można skonstruować estymator ELBO dla całego zbioru danych (o wielkości N) oparty o losowy podzbiór (ang. *minibatch*) o wielkości M:

$$\widetilde{\mathcal{L}}^M(q, X) = \frac{N}{M}\sum_{i=1}^{M} \widetilde{\mathcal{L}}(q, x_i) \tag{1.8}$$

który następnie można różniczkować i użyć razem z metodami gradientowymi jak SGD do optymalizacji θ oraz ϕ.

Auto-enkodery

Przykładem konkretnego modelu implementującego powyższy schemat wariacyjnej inferencji jest następujący *Variational Auto-encoder* (VAE), gdzie:

- rozkład *prior* na zmiennej ukrytej ($p(z)$) przyjmuje się jako $\mathcal{N}(0, \mathbb{I})$, a
- zarówno $p_\theta(x|z)$ jak i *approximate posterior* $q_\phi(z|x)$ reprezentuje się poprzez probabilistyczne sieci neuronowe - rozkłady normalne, których parametry są liczone przez sieci neuronowe.

Żeby wykorzystać estymator SGVB do optymalizacji θ i ϕ, potrzebna jest reparametryzacja $z \sim q_\phi(z|x)$ do $z = g_\phi(\epsilon, x)$, gdzie $\epsilon \sim p(\epsilon)$ jest jakimś niezależnym rozkładem. Dla dowolnego rozkładu normalnego $n \sim \mathcal{N}(\mu, \sigma^2)$ taką reparametryzacją jest translacja $n = \mu + \sigma\epsilon$, gdzie $\epsilon \sim \mathcal{N}(0, 1)$.

W interpretacji auto-enkoderowej, sieć reprezentująca $q_\phi(z|x)$ jest probabilistycznym enkoderem kompresującym przykłady do nisko-wymiarowej przestrzeni ukrytej (*latent space*) z, a sieć $p_\theta(x|z)$ probabilistycznym dekoderem działającym w drugą stronę. Tak jak w zwykłych auto-enkoderach, optymalne kodowanie jest uczone z danych poprzez jednoczesną optymalizację parametrów ϕ i θ poprzez funkcję straty określającą jakość rekonstrukcji. Główną różnicą w VAE jest rozkład $z \sim p(z)$ (*prior*) narzucony na przestrzeń ukrytą, który przekształca go w model generatywny. Można samplować z niego kodowania nowych, nie widzianych wcześniej przykładów, które następnie przekształca się dekoderem do właściwej postaci. W zwykłych auto-enkoderach nie ma żadnych ograniczeń narzuconych na *latent space*, przez co nie da się ręcznie tworzyć znaczących kodowań.

Estymator SGVB w wersji z równiania 1.7 w widoczny sposób odpowiada interpretacji auto-enkoderowej. Człon $\frac{1}{L}\sum_{l=1}^{L} \log p_\theta(x_i|z_{i,l})$ odpowiada jakości rekonstrukcji przykładu, zaś dywergencja $-D_{KL}(q_\phi(z|x_i)||p_\theta(z))$ regularyzuje probabilistyczny enkoder w ten sposób, żeby wyuczył się rozkładu zbliżonego do oczekiwanego na przestrzeni ukrytej.

TODO obrazek

1.3. Powiązane prace

1.3.1. WaveNet

Pierwszą, przełomową pracą generującą dźwięk próbka po próbce za pomocą sieci neuronowych jest powstały w laboratorium DeepMind *WaveNet* [van den Oord et al., 2016a]. Zainspirowany był ówczesnymi osiągnięciami w dziedzinie generowania obrazów. Opublikowany w

tym samym roku PixelRNN [van den Oord et al., 2016b] okazał się generować zadziwiająco wysokiej jakości obrazy autoregresywnie modelując ich rozkład jako ciąg prawopodobieństw warunkowych $p(x_i|x_{i-1}, \ldots, x_0)$, gdzie każdy element x_i to pojedyńczy piksel. Używał do tego dwóch wielowarstwowych LSTM-ów [Hochreiter and Schmidhuber, 1997]. Podobnie do zdyskretyzowanej fali dźwiękowej, modelowany w ten sposób rozkład na obrazkach jest bardzo wysoko wymiarowy (wysokość $\times$ szerokość) i posiada skomplikowane zależności na przestrzeni wszystkich wymiarów.

WaveNet w podobny sposób autoregresywnie faktoryzuje dźwięk, a do wyrażenia rozkładu warunkowego $p(x_i|x_{i-1}, \ldots, x_0)$ (gdzie x_i to pojedyńcza próbka) używa odpowiednio zmodyfikowanych jednowymiarowych warstw konwolucyjnych. Pierwszą modyfikacją jest to, żeby konwolucje rzeczywiście były autoregresywne i polega na "przesunięciu"ich w ten sposób, żeby dany element sekwencji wyjściowej był liczony wyłącznie na podstawie elementów wejściowych o mniejszym indeksie. Drugą modyfikacją są żozciągnięte"konwolucje (*dilated convolutions*) pozwalające WaveNet'owi efektywnie uchwycić zależności na skali wielu tysięcy próbek. Tak jak pokazuje ilustracja TODO, polegają one na tym, że filtr danej konwolucji jest aplikowany do co n-tego elementu (co jest równoważne większemu filtrowi z zerami wstawionymi w odpowiednie miejsca, lecz bardziej efektywne obliczeniowo). Dla sieci złożonej z kilku takich warstw, gdzie żozmiar rozciągnięcia"każdej kolejnej rośnie wykładniczo, obszar pokryty przez taką sieć (ang. *receptive field*) również skaluje się wykładniczo z ilością warstw. Oznacza to, że taka sieć może uczyć się zależności na przestrzeni już sekund, co starczy na przykład do zadań takich jak generacja mowy.

WaveNet można w prosty sposób przekształcić w model warunkowy dokładając do wejścia w odpowiednim miejscu warunkujące dane. W przypadku pojedyńczego, globalnego wektora warunkującego, autorzy dodają go przed funkcją aktywacji, przekształcając go jeszcze przez wyuczoną transformacje liniową. W przypadku ciągu lokalnych informacji warunkujących, zwiększają oni jego częstotliwość za pomocą transponowanych konwolucji do częstotliwości dźwięku a następnie konkatenują przy wejściu do modelu.

Ewaluacja WaveNet'u przeprowadzona została na trzech zadaniach:

- nienadzorowane generowanie mowy,
- nadzorowana synteza tekstu, i
- nienadzorowane generowanie muzyki.

Zestaw nagrań wykorzystany do treningu przy nienadzorowanego generowaniu mowy [Veaux et al., 2017] zawierał 109 mówców, a WaveNet warunkowany był na ich identyfikatorach. Ten sam model był w stanie uchwycić charakterystykę wszystkich mówców naraz, co sugeruje jakiś poziom wspólnej reprezentacji między nimi. Dla nadzorowanej generacji tekstu wykorzystane zostały bazy nagrań Google'a dla północno-amerykańskiego angielskiego oraz mandaryńskiego.

1. WaveNet, generowanie mowy jako pierwsze modele generujące próbka po próbce
2. jakieśtam transkrypcje czy inne klasyfikacyjne taski
3. Melnety czy inne takie na spektrogramach
4. te wszystkie rzeczy na wavenetach do TTSu

1.3.2. SampleRNN

1.3.3.

1.4. Normalizing Flows

normalizing flows coto czemu oraz w zastosowaniu:

1. do estymowania gęstości,
2. do parallel wavenetów może?
3. lepszych VAE
4. i coś tam jeszcze chyba było

1.5. DDSP

no tak, czym są komponenty ddsp (opisy matematyczne?), co to jest INDUCTIVE BIAS, i omówienie ich zajebistych wyników i na koniec że połączenie z VAE to ez win

Rozdział 2

Architektura

ta

1. no te enko-deko-dery ddspowe
2. liczenie stftów
3. framerate, upsampling do komponentów
4. podłączenie komponentów

Rozdział 3

Eksperymenty oraz ewaluacja

uhhhhhh

Rozdział 4

Podsumowanie i wnioski

ciężko

Bibliografia

Rameen Abdal, Yipeng Qin, and Peter Wonka. Image2stylegan: How to embed images into the stylegan latent space?, 2019.

Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations, 2020.

Rodrigo Castellon, Chris Donahue, and Percy Liang. Codified audio language modeling learns useful representations for music information retrieval, 2021.

A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society. Series B (Methodological)*, 39(1):1–38, 1977. ISSN 00359246. URL `http://www.jstor.org/stable/2984875`.

Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: A generative model for music, 2020.

Chris Donahue, Julian McAuley, and Miller Puckette. Adversarial audio synthesis, 2019.

Jeff Donahue, Sander Dieleman, Mikołaj Bińkowski, Erich Elsen, and Karen Simonyan. End-to-end adversarial text-to-speech, 2021.

Alexandre Défossez. Hybrid spectrogram and waveform source separation, 2021.

Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Douglas Eck, Karen Simonyan, and Mohammad Norouzi. Neural audio synthesis of musical notes with wavenet autoencoders, 2017.

Jesse Engel, Kumar Krishna Agrawal, Shuo Chen, Ishaan Gulrajani, Chris Donahue, and Adam Roberts. Gansynth: Adversarial neural audio synthesis, 2019.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014. URL `https://proceedings.neurips.cc/paper/2014/file/5ca3e9b122f61f8f06494c97b1afccf3-Paper.pdf`.

Google. Nsynth super, 2018. URL `https://nsynthsuper.withgoogle.com/`.

Google. Magenta studio, 2021. URL `https://magenta.tensorflow.org/studio`.

Alex Graves. Generating sequences with recurrent neural networks, 2014.

Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. Enabling factorized piano music modeling and generation with the maestro dataset, 2019.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9: 1735–80, 12 1997. doi: 10.1162/neco.1997.9.8.1735.

Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Curtis Hawthorne, Andrew M Dai, Matthew D Hoffman, and Douglas Eck. Music transformer: Generating music with long-term structure. *arXiv preprint arXiv:1809.04281*, 2018.

Nal Kalchbrenner and Phil Blunsom. Recurrent continuous translation models. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1700–1709, Seattle, Washington, USA, October 2013. Association for Computational Linguistics. URL `https://aclanthology.org/D13-1176`.

Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks, 2019.

Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan, 2020.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2014.

Kundan Kumar, Rithesh Kumar, Thibault de Boissiere, Lucas Gestin, Wei Zhen Teoh, Jose Sotelo, Alexandre de Brebisson, Yoshua Bengio, and Aaron Courville. Melgan: Generative adversarial networks for conditional waveform synthesis, 2019.

John D. Lafferty, Andrew McCallum, and Fernando Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *ICML*, 2001.

Soroush Mehri, Kundan Kumar, Ishaan Gulrajani, Rithesh Kumar, Shubham Jain, Jose Sotelo, Aaron Courville, and Yoshua Bengio. Samplernn: An unconditional end-to-end neural audio generation model, 2017.

Mirco Ravanelli, Philemon Brakel, Maurizio Omologo, and Yoshua Bengio. Light gated recurrent units for speech recognition. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(2):92–102, Apr 2018. ISSN 2471-285X. doi: 10.1109/tetci.2017.2762739. URL `http://dx.doi.org/10.1109/TETCI.2017.2762739`.

Shreeviknesh Sankaran, Sukavanan Nanjundan, and G. Paavai Anand. Anyone gan sing, 2021.

C.E. Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21, 1949. doi: 10.1109/JRPROC.1949.232969.

Jonathan Shen, Ruoming Pang, Ron J. Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, RJ Skerry-Ryan, Rif A. Saurous, Yannis Agiomyrgiannakis, and Yonghui Wu. Natural tts synthesis by conditioning wavenet on mel spectrogram predictions, 2018.

S. S. Stevens, J. Volkmann, and E. B. Newman. A scale for the measurement of the psychological magnitude pitch. *The Journal of the Acoustical Society of America*, 8(3):185–190, 1937. doi: 10.1121/1.1915893. URL `https://doi.org/10.1121/1.1915893`.

Ilya Sutskever, Geoffrey Hinton, and Graham Taylor. The recurrent temporal restricted boltzmann machine. In *Proceedings of the 21st International Conference on Neural Information Processing Systems*, NIPS'08, page 1601–1608, Red Hook, NY, USA, 2008. Curran Associates Inc. ISBN 9781605609492.

Ilya Sutskever, James Martens, and Geoffrey Hinton. Generating text with recurrent neural networks. In *Proceedings of the 28th International Conference on International Conference on Machine Learning*, ICML'11, page 1017–1024, Madison, WI, USA, 2011. Omnipress. ISBN 9781450306195.

Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio, 2016a.

Aaron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks, 2016b.

Christophe Veaux, Junichi Yamagishi, and Kirsten MacDonald. Cstr vctk corpus: English multi-speaker corpus for cstr voice cloning toolkit. 2017.

Zhengwei Wang, Qi She, and Tomas E. Ward. Generative adversarial networks in computer vision: A survey and taxonomy, 2020.

Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks, 2020.